

MICROSOFT AGENTS LEAGUE @ AISF 2026

CODEBASE ATLAS

Technical Documentation

Google Maps for any codebase — an AI guide flies you through the code

Author

S T P Hayavadan

Creative Apps Track · Foundry IQ

Project: Codebase Atlas | Language: Python + TypeScript

Stack: FastAPI · networkx · React · Sigma.js · WebGL · Microsoft Foundry IQ

Version 1.0 · June 2026

Table of Contents

1. Introduction	3
2. Overview	3
3. Problem Statement	4
4. Business Challenge & Impact	4
5. Solution Overview	5
6. Technical Architecture	6
7. User Flow Architecture (Flowchart)	8
8. Future Scope	9
9. Conclusion	9
10. Vote of Thanks	10

1. Introduction

Codebase Atlas is an interactive system that turns any Python repository into a living, zoomable map and then guides a developer through it with an AI narrator. Think of it as “Google Maps for a codebase”: pull the camera back to see packages as glowing continents, zoom in to reveal files as cities, classes as districts, functions as streets, and methods as individual buildings. An AI guide narrates an opening establishing shot of the whole repository, then explains every place on the map in plain language, spoken aloud.

The project was built for the **Microsoft Agents League @ AISF 2026** hackathon under the **Creative Apps** track, and integrates **Microsoft Foundry IQ** as its intelligence layer — satisfying the event requirement that every submission use at least one Microsoft IQ service. Foundry IQ powers the narration, the guided-tour sequencing, and the question-answering navigation, while all structural analysis remains deterministic and model-free.

Document purpose

This document is the technical reference for Codebase Atlas: its motivation, architecture, data flow, and the design decisions behind each subsystem. It is intended for engineers, reviewers, and judges who want to understand how the system is built and why.

2. Overview

At a high level, Codebase Atlas is split into two cleanly separated halves that meet at a single JSON contract:

The deterministic half (structure)

Parsing, graph construction, and layout are entirely LLM-free. Given a repository path, the backend walks the source tree, parses each file with Python’s native `ast` module, builds a directed graph of packages, modules, classes, functions and methods (with *contains*, *imports* and *calls* edges) using `networkx`, computes stable 2D map coordinates with a seeded spring layout, and serializes everything into one self-contained `graph.json` document. This half is fast, reliable, and cached.

The narrative half (intelligence)

Microsoft Foundry IQ is used *only* for narrative: writing a warm explanation for every node, designing the teaching order of the guided tour, and answering free-form questions by selecting the best node on the map to fly the camera to. Each model call carries only the relevant slice of the graph, so requests stay fast, cheap, and on-topic. A background build job pre-generates all narration up front, so the interactive experience never stalls waiting on the model.

Key facts at a glance

Aspect	Detail
Input	Any public or private Git repository (Python source)
Structural engine	Python <code>ast</code> + <code>networkx</code> (deterministic, cached, LLM-free)
Intelligence layer	Microsoft Foundry IQ — FAST model (narration, Q&A) + SMART model (tour)
Backend	FastAPI, background BuildJob, in-memory artifact cache

Frontend	React + TypeScript + Sigma.js (WebGL), Web Speech API for TTS
Core contract	A single graph.json: nodes, edges, layout, stats

3. Problem Statement

Understanding an unfamiliar codebase is one of the most consistently painful tasks in software engineering. The core difficulties are structural:

- **Cold start.** A new engineer is handed a folder tree of hundreds of files and no indication of where the program's story actually begins.
- **No living map.** READMEs and architecture docs drift out of date almost immediately. The real structure lives implicitly, scattered across imports and call sites.
- **Tribal knowledge.** The mental model of how a system fits together usually lives in the heads of a few senior engineers, and walks out the door when they do.
- **Context tax.** Every "where is X handled?" becomes a manual grep-and-read expedition, interrupting both the newcomer and whoever they ask.

Fundamentally, code *structure* is a graph, but we almost always consume it as flat text. Humans navigate by space and narrative — a map and a guide — not by an alphabetised list of filenames. Codebase Atlas exists to close that gap.

4. Business Challenge & Impact

The business challenge is the direct cost of the problem above: onboarding is slow, senior engineering time is consumed by hand-holding, and knowledge is fragile. Codebase Atlas addresses each of these with measurable impact.

Faster onboarding

A new hire can be dropped into a narrated, guided flight through the system instead of a folder tree, compressing the "where do I even start" phase from weeks toward hours.

Reduced context tax & knowledge retention

By answering "where is X?" with a camera that flies to the exact node, the tool removes routine interruptions to senior staff. Because the map and narration are generated artifacts, the system's mental model is captured in a durable, shareable form rather than living only in people's heads.

Code review, audits, and open-source adoption

Reviewers and security auditors gain a guided index into unfamiliar areas. For open-source projects, a living explainer lowers the barrier to a first contribution — newcomers can see the lay of the land before touching a single line.

5. Solution Overview

Codebase Atlas delivers an end-to-end experience: paste a repository URL, watch the system pre-generate everything, then explore a cinematic WebGL map narrated by an AI guide. The solution is organised into the deterministic structural pipeline and the Foundry IQ narrative layer.

5.1 Structural pipeline (LLM-free)

- **ingest.py** — walks the repository, skips noise (virtualenvs, caches, build artifacts) and collects only authored Python source files, deriving a dotted module path and package for each.

- **ast_parser.py** — parses each file with Python's native `ast`, extracting classes, functions, methods, signatures, docstrings, decorators, the calls each function makes, source slices, and a crude complexity score. Syntax errors degrade gracefully to an empty module rather than crashing the run.
- **graph_builder.py** — assembles a `networkx` directed graph with a node taxonomy tagged by *level* for semantic zoom (package → module → class/function → method) and three edge kinds: *contains*, *imports* (intra-repo only), and *calls* (resolved within the repo, best-effort). It also heuristically detects likely entry points.
- **layout.py** — assigns stable 2D coordinates: packages on a ring, modules force-directed within each package, and symbols in radial satellite rings around their module, with methods orbiting their class. Seeded so the map never jumps between runs.
- **serialize.py** — emits the single `graph.json` document with per-node colors and sizes (dark-cinematic palette keyed by kind) plus repo-level stats.

5.2 Narrative layer (Microsoft Foundry IQ)

- **narrate.py** — produces a warm, spoken-style 2–4 sentence explanation for each node and a establishing-shot overview for the whole repo, feeding the model only the node's source plus a compact summary of its graph neighbourhood.
- **tour.py** — uses the SMART Foundry model to design the learning order of 6–12 tour stops (entry point → core abstractions → supporting cast), returning structured JSON with a fallback based on entry points and fan-in.
- **kb.py** — a Foundry IQ retrieval engine that ranks the most relevant nodes for a natural-language query from a compact catalog.
- **qa_agent.py** — the signature feature: given a question, it selects the single best node, writes a grounded spoken answer, and returns the node id so the frontend flies the camera there.
- **build_job.py** — orchestrates the whole lifecycle in a background thread and pre-generates the overview plus an explanation for every node in parallel (thread pool), exposing live progress.

6. Technical Architecture

The diagram below shows how data moves through the system, from a raw repository to the rendered map. The structural pipeline produces `graph.json`; the background build job enriches it with Foundry IQ narration and a designed tour; FastAPI serves all artifacts from an in-memory cache; and the React/Sigma.js frontend renders the WebGL map and flies the camera.

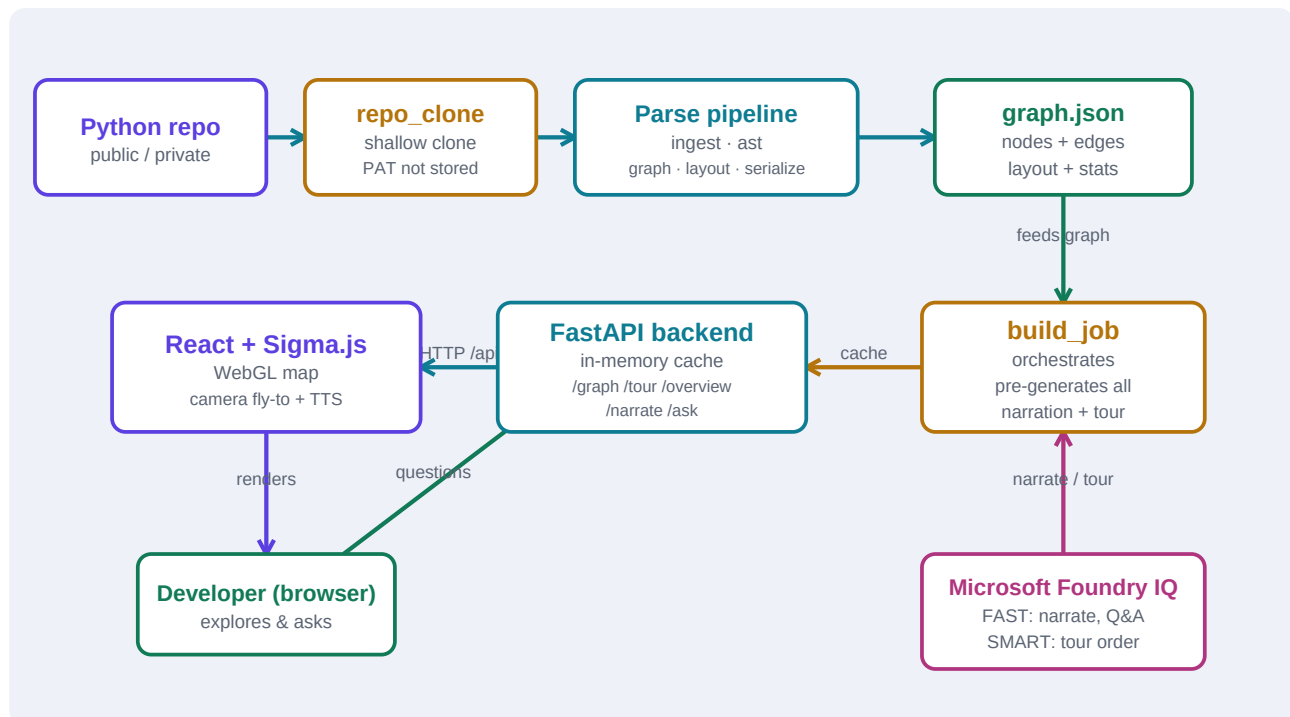


Figure 1 — Technical architecture & data flow. Clone → deterministic parse to `graph.json` → Foundry IQ enrichment via `build_job` → FastAPI cache → WebGL frontend.

Design principle: separation of concerns

Structure is computed deterministically and cached; intelligence is layered on top and never blocks the critical path. Because narration is fully pre-generated before the dive begins, every step of the interactive experience is an instant in-memory read. Secrets (the Foundry key and the repository PAT) stay server-side; the browser never sees a credential, and the PAT lives only in the argv of the git clone subprocess.

7. User Flow Architecture

The diagram below is the user's journey as a flowchart — from pasting a repository URL through to an open-ended, interactive exploration of the map. The experience is phase-based: *home* → *building* → *atlas*, with the Q&A loop keeping the session interactive indefinitely.

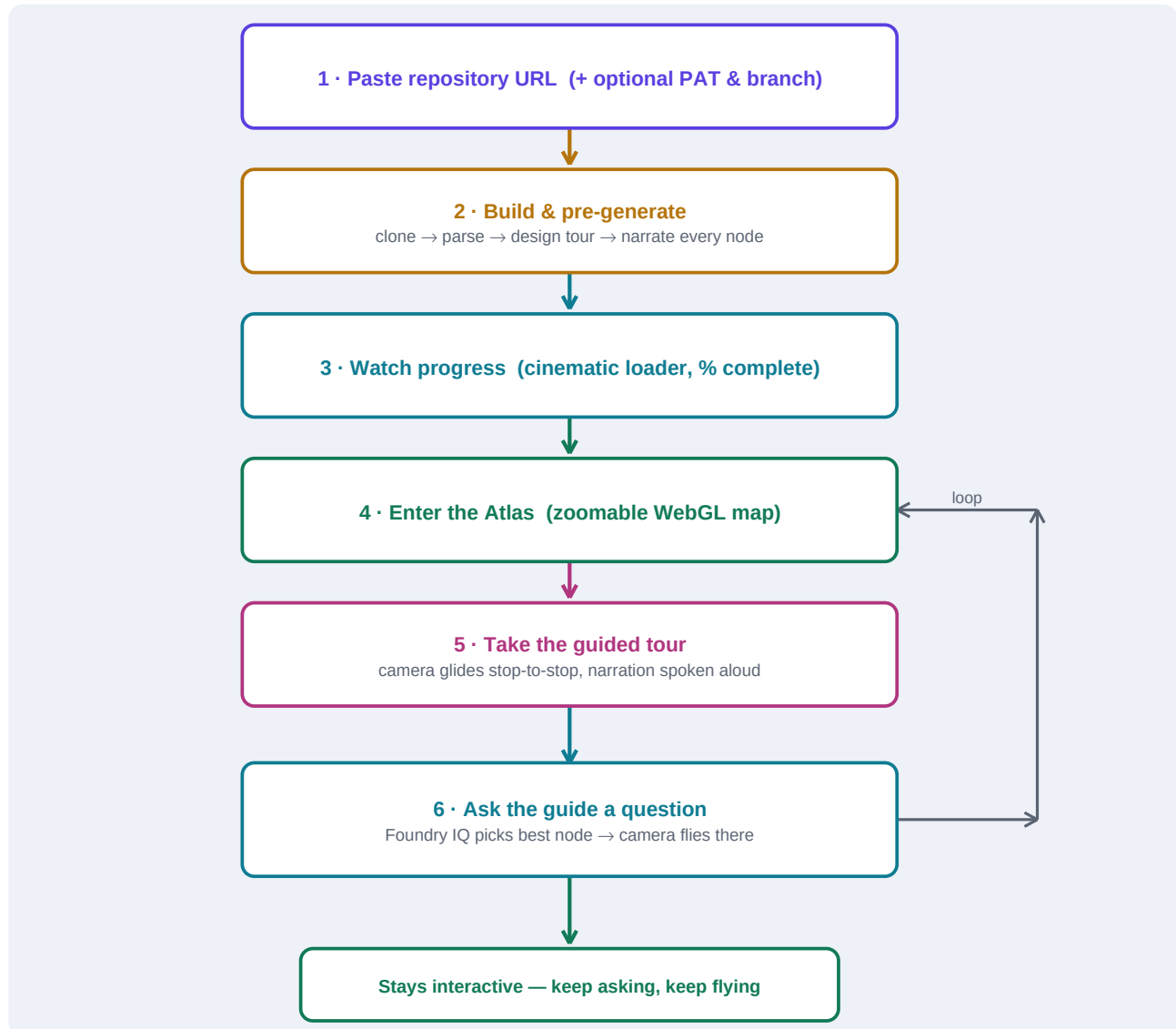


Figure 2 — User flow as a flowchart. Phases: *home* → *building* → *atlas*. The Q&A step loops back into the map, so exploration continues for as long as the user wants.

8. Future Scope

- **More languages.** The graph schema is language-agnostic; JavaScript/TypeScript, Go, and Java can be added behind the same `/api/graph` contract without touching the frontend.
- **Premium narration voices.** Swap the browser's free Web Speech API for Azure Speech to get richer, multilingual, more natural voices.
- **Live diff tours.** Narrate a pull request — “here is what changed and why it matters” — turning Atlas into a code-review companion.
- **Team annotations.** Let humans pin notes on nodes, blending AI narration with curated tribal knowledge.
- **Deeper retrieval.** Extend Foundry IQ retrieval over commit history and issues, not just the current source tree.
- **Embeddable widget.** Ship the Atlas as a drop-in living explainer for any project's documentation site.

9. Conclusion

Codebase Atlas reframes the universal pain of “understanding a new repository” as a guided, cinematic flight. Its deterministic parsing layer provides a fast, reliable, and cached map of the code, while Microsoft Foundry IQ gives that map a voice and a sense of direction — explaining, ordering, and navigating. The clean separation between structure and narrative keeps the system both trustworthy and responsive: the map is always correct, and the experience never stalls because every explanation is generated up front.

The result answers the oldest question in software engineering — “where do I even start?” — not with a wall of text, but with a map, a tour, and a guide who will fly you straight to the answer.

10. Vote of Thanks

Sincere thanks to the **Microsoft team** for organising the **Agents League @ AISF 2026** hackathon. The live coding battles on Microsoft Reactor, the welcoming Discord community, and the opportunity to build with **Foundry IQ** made this an exceptional experience. Foundry IQ is the intelligence layer that gave this project its voice, and the event's structure made building under a deadline genuinely enjoyable.

Thank you to the judges and mentors for their time and feedback, and to everyone who keeps these communities thriving. This project stands on the shoulders of excellent tools and a generous community.

[Codebase Atlas](#) · [S T P Hayavadan](#) · [Microsoft Agents League @ AISF 2026](#) · [Creative Apps](#) · [Foundry IQ](#)